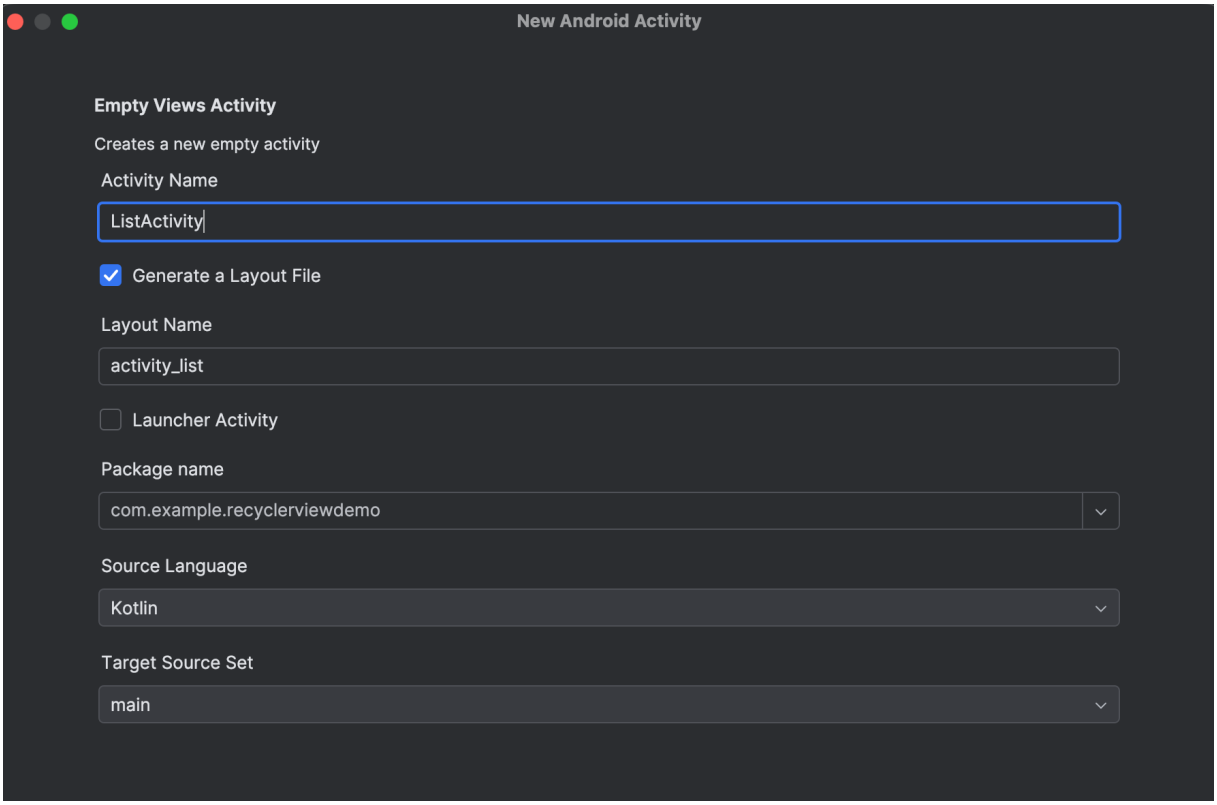
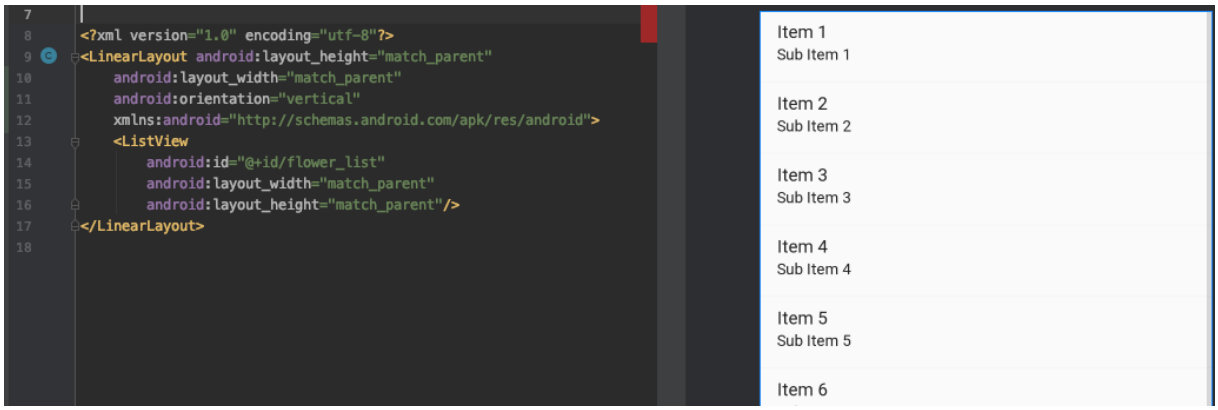
Android App Development – Working with lists

- One of the most commonly used "views" in mobile applications is the list - this is due to the possibility of scrolling it which allows the presentation of many information (items) on a limited screen. We will try to work with such a component for a moment.
- Let’s create a new Empty Views Acrivity Project with API 34 / 35 / 36 as the minimum SDK
- We add a new empty activity to the project. Let’s call it ListActivity, we generate a layout for it, it will not be a startup activity.



- We add a button to our main activity that will take you to the activity just added. Let's assume that the list will present a list of flowers - so set the button caption to "show flower list"
- We go to an xml file that defines the appearance of the activity just added. We remove the ConstraintLayout added by default and replace it with LinearLayout with vertical orientation, and with height and width such as "parent".
- Inside LinearLayout we add a ListView element with width and height such as "parent" and id set to @+id/flower_list. So finally the layout of this new activity looks as follows:

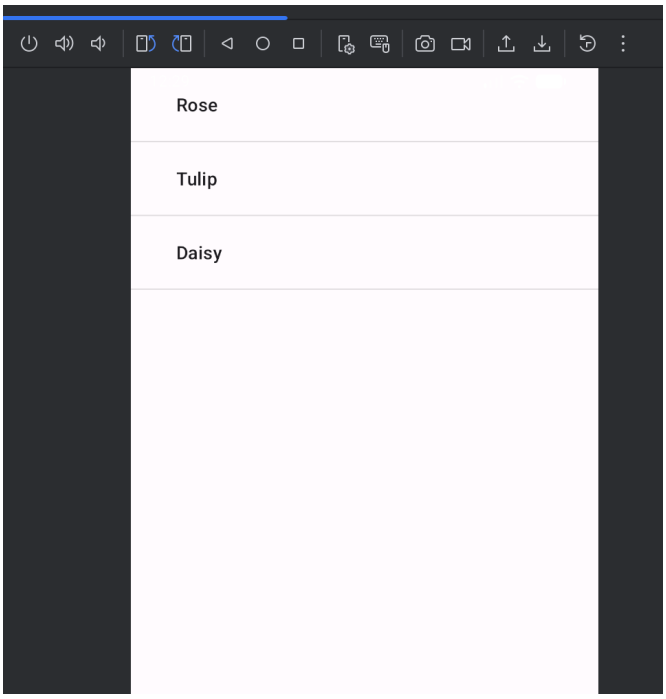
- 

- Turn on the view binding in your gradle file:
  - 

- Let’s go to the class that defines our newly added activity (ListActivity) and modify it as below:

```
2 Usages
class ListActivity : AppCompatActivity() {
    4 Usages
    private val listOfFlowers = ArrayList<String>()
    3 Usages
    private lateinit var binding: ActivityListBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityListBinding.inflate(layoutInflater)
        setContentView(binding.root)

        initFlowerList()
        var adapter = ArrayAdapter( context = this,
            resource = android.R.layout.simple_expandable_list_item_1,
            objects = listOfFlowers)

        binding.flowerList.adapter = adapter
    }

    1 Usage
    private fun initFlowerList() {
        listOfFlowers.add("Rose")
        listOfFlowers.add("Tulip")
        listOfFlowers.add("Daisy")
    }
}
```

- In other words, we create a list of flowes (globally for the class because it may still be useful). Then in the onCreate method - we call an auxiliary method in which we put some sample names into the list - I extracted it into a separate method - for readability. Then I create an ArrayAdapter which "maps" each element of our list to a single "item" of the list. As an item, I used the built-in element available on Android, called simple_expandable_list_item_1.
- Run and test the application. The result should be as below:

- 

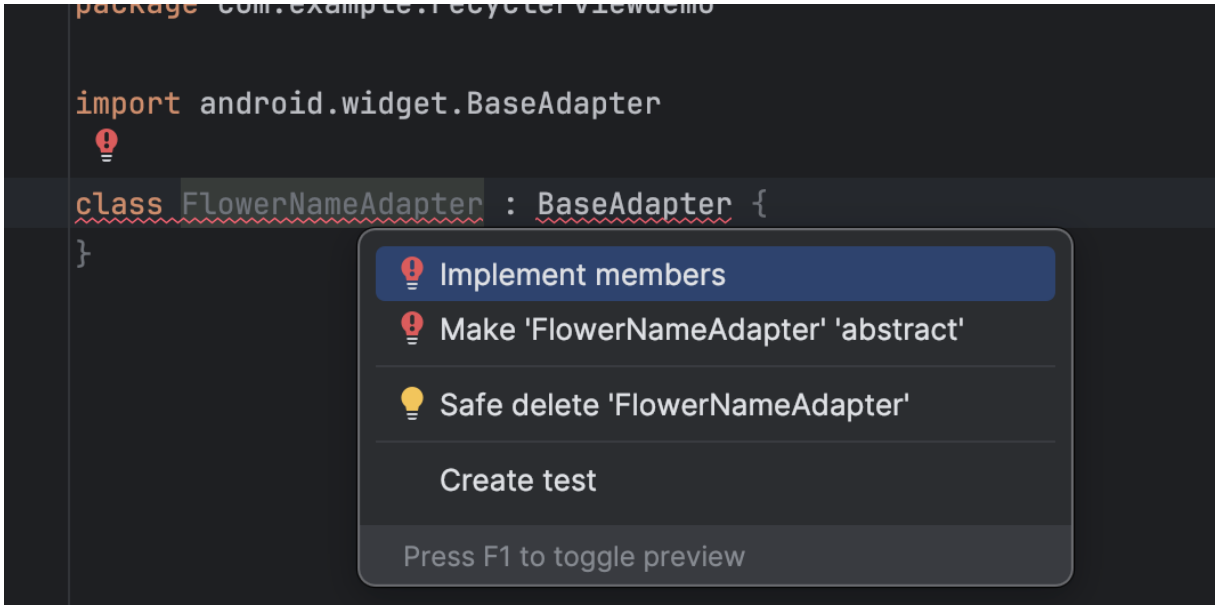
- Next, instead of using the built-in definition of each of the table's rows, let's try to define how such a row should look like. And so we add a new layout file to the project (right click on res/ layout -> New -> XML -> Layout XML File. Let's set the name to flower_list_item. As the root element we use the RelativeLayout.

- 

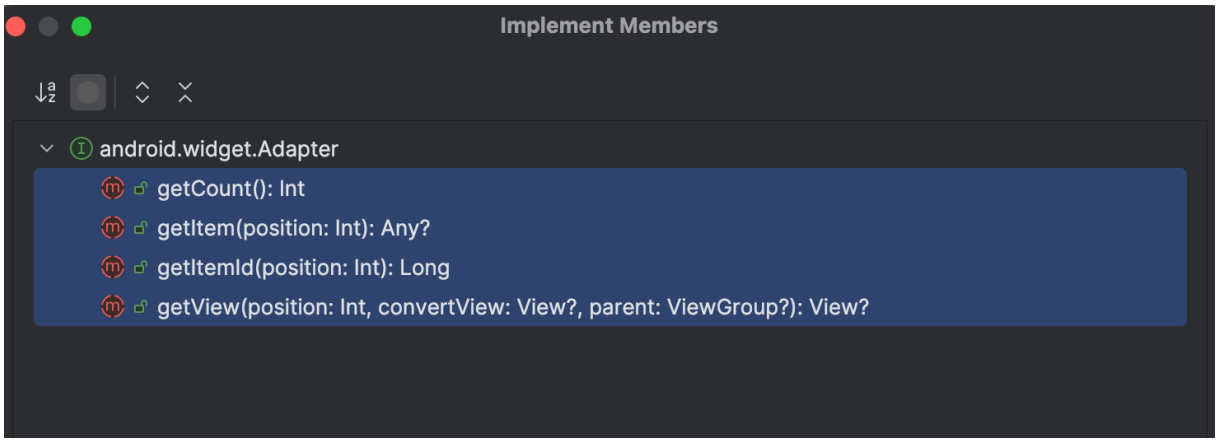
- To start with, the only thing we need in this layout is the TextView element. When it comes to "beautifying" the item it is absolutely up to you. Notice that it is important to set the id for this field - let's set it to @+id/flower_name. In my case the definition of the layout is as follows:

```xml
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:id="@+id/flower_name"
        android:background="@android:color/holo_green_light"
        android:textSize="25sp"
        android:textStyle="bold"
        android:paddingStart="5sp"
        android:paddingLeft="5sp"
        android:paddingRight="5sp"
        android:paddingEnd="5sp"/>
</RelativeLayout>
```

- Next - we need to create our own adapter that will intermediate between our list of flowers, the ListView and its individual items. To do that, we add a new class to the project – so do the right click on the sourcecode folder -> New Kotlin class/file. Let's call the class as FlowerNameAdapter and this class will inherit from the BaseAdapter class.
- Nexy, being with the cursor on the class name press alt + enter and select “implement members” from the contextual menu



- And we choose all four methods available for implementation

- 

- In addition, we add two fields to the class: context, and a list of flowers, so we need also the constructor to set these fields as below:
- ```
class FlowerNameAdapter : BaseAdapter {
    1 Usage
    lateinit var context: Context
    1 Usage
    lateinit var flowerNames: ArrayList<String>
    constructor(context: Context,flowerNames: ArrayList<String>): super() {
        this.context = context
        this.flowerNames = flowerNames

    }

    override fun getCount(): Int {
        TODO(reason = "Not yet implemented")
    }
```
- And now we proceed with the implementation of all methods in this class.
- getCount should return the size of the list with flower names:
- ```
    override fun getCount(): Int {
        return flowerNames.size
    }
```
- Let getItem return the name of the flower under the indicated position:

```
override fun getItem(position: Int): Any {
    return flowerNames[position]
}
```

- getItemID should return the position converted to Long

```
override fun getItemId(position: Int): Long {
    return position.toLong()
}
```

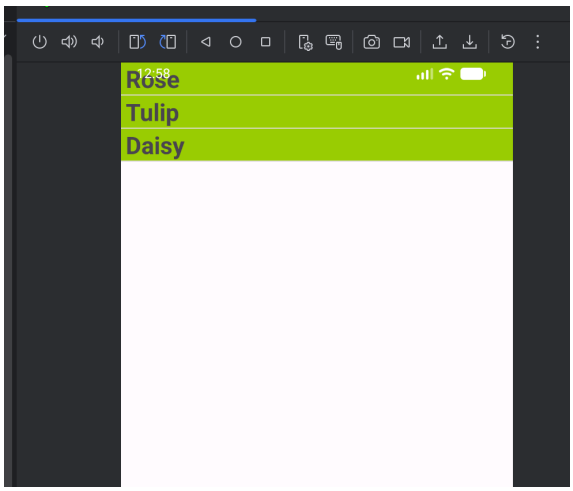
- And the getView should be implemented as below:

```
override fun getView(
    position: Int,
    convertView: View?,
    parent: ViewGroup?
): View? {

    val binding: FlowerListItemBinding
    val itemView = if (convertView == null) {
        binding = FlowerListItemBinding.inflate(
            inflater = LayoutInflater.from(context),
            parent,
            attachToParent = false
        )
        binding.root.tag = binding
        binding.root
    } else {
        binding = convertView.tag as FlowerListItemBinding
        convertView
    }
    binding.flowerName.text = getItem(position) as CharSequence?
    return itemView
}
```

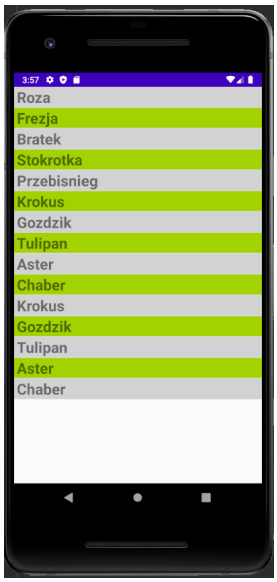
- Now we go to the ListActivity class and replace the adapter used so far with the one we just implemented:

```
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityListBinding.inflate(layoutInflater)
        setContentView(binding.root)

        initFlowerList()
//        var adapter = ArrayAdapter(this,
//            android.R.layout.simple_expandable_list_item_1,
//            listOfFlowers)
//
//        binding.flowerList.adapter = adapter

        var flowerAdapter = FlowerNameAdapter( context = this, flowerNames = listOfFlowers)
        binding.flowerList.adapter = flowerAdapter
    }
```

- I have intentionally commented on the old lines of code so that you can see what has been changed.
- Run and test applications, and you should get something similar to:



- Small practice: Modify the solution so that every second flower name is displayed on a different background:

- 

- Now, let's try for each flower to add its "photo" to the list.
- Let's start by copy-paste'ing the file that defines the layout of each line of our list (for me the file flower_list_item.xml) - when duplicating, set its name to flower_list_item_1.xml.
- Let's modify this new layout by adding ImageView located to the right of the flower name. For me, the definition after the changes is as follows:

```xml
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:id="@+id/flower_name"
        android:background="@android:color/holo_green_light"
        android:textSize="25sp"
        android:textStyle="bold"
        android:paddingStart="5sp"
        android:paddingLeft="5sp"
        android:paddingRight="5sp"
        android:paddingEnd="5sp"/>

    <ImageView
        android:id="@+id/flower_image"
        android:layout_width="48sp"
        android:layout_height="48sp"
        android:layout_alignParentTop="true"
        android:layout_alignParentRight="true"
        android:paddingRight="5sp"/>

</RelativeLayout>
```

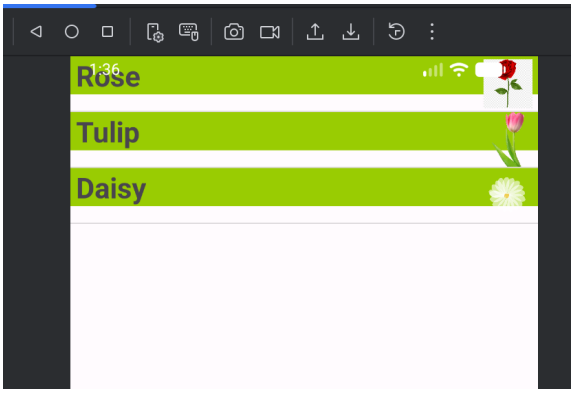
- The changes we need to make is the addition of a list that stores the id's of flower pictures. As a consequence, we are expanding the class constructor with this additional parameter, and in the getView method, apart from setting the flower name, we also set a flower photo. The rest can stay as it was. My implementation looks like this

```
class FlowerNameAdapter : BaseAdapter {
    lateinit var flowerImages: ArrayList<Int>

    constructor(context: Context, flowerNames: ArrayList<String>, flowerImages: ArrayList<Int>) : super() {
        this.context = context
        this.flowerNames = flowerNames
        this.flowerImages = flowerImages
    }
    override fun getCount(): Int {...}
    1 Usage
    override fun getItem(position: Int): Any? {...}
    override fun getItemId(position: Int): Long {...}
    override fun getView(
        position: Int,
        convertView: View?,
        parent: ViewGroup?
    ): View? {

        val binding: FlowerListItem1Binding
        val itemView = if (convertView == null) {
            binding = FlowerListItem1Binding.inflate(
                inflater = LayoutInflater.from(context),
                parent,
                attachToParent = false
            )
            binding.root.tag = binding
            binding.root
        } else {
            binding = convertView.tag as FlowerListItem1Binding
            convertView
        }
        binding.flowerName.text = getItem(position) as CharSequence?
        binding.flowerImage.setImageResource(flowerImages[position])

        return itemView
    }
```

- Now let's add some pictures of flowers to the drawable (you will find on the web - remember that the files must be in .png format and that the names cannot contain special characters, capital letters etc.)
- Now in the ListActivity class we add a list for storing the id of individual photos and then we add photos to the list, and we "switch" to the adapter we just created:

```
2 Usages
class ListActivity : AppCompatActivity() {
    1 Usage
    private val flowerNames = arrayListOf(
        "Rose",
        "Tulip",
        "Daisy"
    )
    1 Usage
    private val flowerImages = arrayListOf(
        R.drawable.rose,
        R.drawable.tulip,
        R.drawable.daisy
    )
    3 Usages
    private lateinit var binding: ActivityListBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityListBinding.inflate(layoutInflater)
        setContentView(binding.root)

        var flowerAdapter = FlowerNameAdapter( context = this, flowerNames, flowerImages)
        binding.flowerList.adapter = flowerAdapter
    }

}
```

- Run and test applications. The effect should be similar to the below one:



- ▪ When completed pls prepare the short videocast presenting how your app is implemented and how it works. The video should be uploaded to the upel platform